

Insight Ladder — Day 2

Which pairs count?

Priyanka | September 22, 2026 | Medium relative to this week | 120 minutes

Main problem

Given an unsorted integer array and inclusive bounds $L \leq R$, count index pairs $i < j$ satisfying $L \leq a[i] + a[j] \leq R$. Equal values at different indices represent different pairs. $n \leq 200,000$; values and bounds lie between $-1,000,000,000$ and $1,000,000,000$. Require $O(n \log n)$ time; sorting a copy is allowed. In C, use 64-bit arithmetic for sums and counts. State the cost of sorting and its storage separately from your scan.

Examples and checks: $[1, 1, 2, 3]$ with $[L, R] = [3, 4]$ gives 4. $[2, 2, 2]$ with $[4, 4]$ gives 3. Empty and singleton arrays give 0. Also test negative numbers, $L = R$, and no valid pairs.

Optional variation

Count distinct unordered VALUE pairs instead of index pairs, in $O(n \log n)$ time. A pair (x, x) is allowed only if the input contains at least two copies of x . For $[1, 1, 2, 3]$ and $[3, 4]$, the answer is 2; for $[2, 2, 2]$ and $[4, 4]$, it is 1. Explain why simply removing duplicates and running the original routine is insufficient.

Before reading hints: Can one exact range be expressed using two easier counting questions? Why can your algorithm count a whole block of pairs without listing them?

Your 120 minutes

- 0–10: closed-notes recall of one invariant or failed assumption from the previous session.
- 10–100: solve the main problem. Write a direct solution, identify repeated work, state a claim, try to break it, then implement. Attempt the optional variation only when the main solution is secure; it stays inside these 90 minutes.
- 100–120: stop coding, explain correctness and cost, review evidence, and log partial work. A full solution read during review does not turn an unfinished timed attempt into a solve.
- Use C11 OR Python 3 consistently. After 15 minutes without a useful new step, record the blocker and request one hint or continue. Record the time and level of every hint or appendix consultation.

Submit your evidence

Submit code, tests, the key claim and its justification, runtime/space, minutes spent, and hints used. Main status: independent solve / assisted solve / partial / unattempted. Insight: independent / after small hint / after method supplied. Track variation coding or reasoning separately; a counterexample is not another code solve. Count a main solve only when working code meets the contract and you can justify it. There is no completion quota.

Solution appendix

Open only after an attempt; record what you read and when. Read one hint at a time. The explanation below supplies the method and cannot count as independent discovery.

Progressive hints

1. First solve an easier question: how many pairs have sum at most a threshold?
2. In sorted order, if the smallest remaining value plus the largest is small enough, which other partners for that smallest value also work?
3. Subtract the number of pairs with sum strictly below L from the number with sum at most R . Use separate strict/non-strict comparisons instead of relying on $L-1$.

Main solution reasoning

Sort. To count pairs with sum $\leq T$, place l at 0 and r at $n-1$. If $a[l]+a[r] \leq T$, all $r-l$ partners from $l+1$ through r work for l ; add $r-l$ and increment l . Otherwise no pair using r with any remaining left endpoint can work, so decrement r . Each step eliminates exactly the pairs just counted or proved invalid. Change $\leq$ to $<$ to count sums strictly below L . Subtract $\text{countLess}(L)$ from $\text{countAtMost}(R)$. Each pointer moves $O(n)$ times. Total time is $O(n \log n)$, with $O(1)$ scan storage; a Python sorted copy uses $O(n)$ storage. In C report the actual sorting implementation storage.

Variation review

Compress into sorted distinct values v and their frequencies. Count distinct-value pairs with different values using the same two counting scans over v . Then add one diagonal pair $(v[i], v[i])$ for each frequency ≥ 2 whose doubled value is in range. Deduplication alone loses these valid diagonals. Compression and scans cost $O(n)$ after sorting; $O(n)$ storage is allowed.

Exit ticket and repair

Can one exact range be expressed using two easier counting questions? Why can your algorithm count a whole block of pairs without listing them?

If the explanation is still unclear: Enumerate the six index pairs of $[1,1,2,3]$ and compare them with its distinct value pairs.

Log: main attempted __; independent solve __; assisted solve __; partial __; variation status __; insight assistance __; minutes to first useful claim __; total minutes __.